UTPORTO
Universidade Técnica do Porto

NORTE 2030
Programa Regional do Norte

PORTUGAL 2030

Cofinanciado pela União Europeia

~/tweb/aula-08

GitHub Pages and accessibility_

Lesson 8 · 27 Oct 2026

CTeSP DTAM
Tecnologias Web

Lecturer
Gonçalo Reais

// review

Last lesson in brief

- A **commit** records a version of the project, with the author, the date and a message that says what changed.
- `git add` chooses the changes that go into **staging**; `git commit` saves them in the repository.
- `git push` sends to GitHub the commits that are still only on the computer.
- Git records the history on the computer; **GitHub** keeps a copy of the repository on the Internet.
- `README.md`, written in Markdown, is the repository's front page.

// objectives

Lesson objectives

01

Work in a **branch** and join it into `main` with `git merge`, resolving conflicts.

02

Publish the site with **GitHub Pages** and fix the common publishing mistakes.

03

Explain what web **accessibility** is and the four principles of WCAG.

04

Test a page with the **keyboard** only and with a **screen reader**.

// branches

Experimenting and working in parallel

In both cases, work is going on while the **version that works**, on `main`, has to stay intact.

```
Documents/
├── meu-site/
├── meu-site-copy/
└── meu-site-experiment/
```

experimenting

Ana wants to test a new version of the "about me" page, which might not work out. Copying the folder means going back to the file names from lesson 7.

```
Ana changes index.html → main
Rui changes contacto.html → main
at the same time, in the same repository
```

working as a team

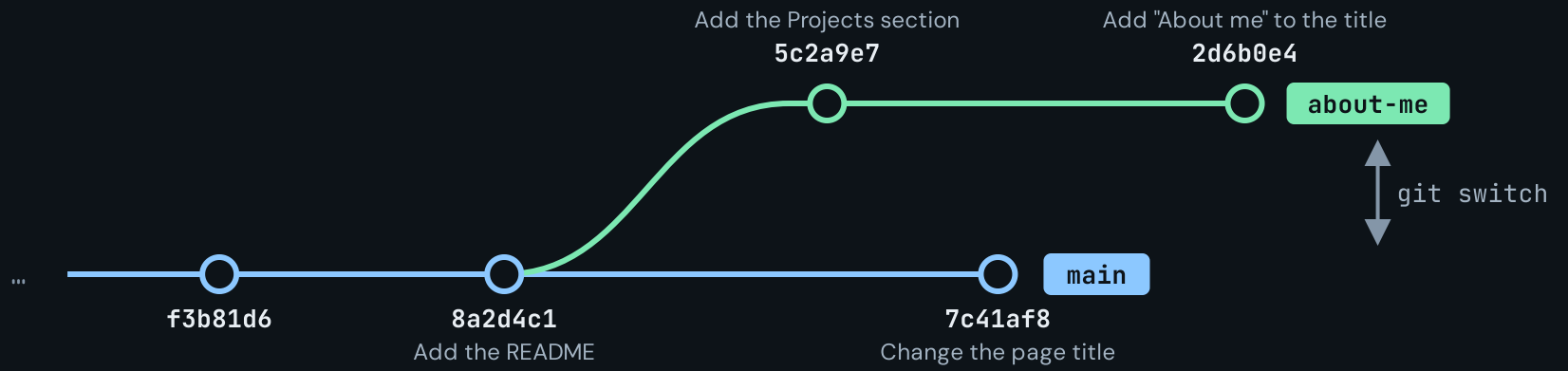
Ana changes the "about me" page while Rui changes the form. On one `main`, half-done work from one gets mixed up with the other's.

How can you work in parallel without touching the version that works?

// branches

What a branch is

A **branch** is a parallel line of work, with its own commits. Inside, it is a label that points to a commit, like `main`. What is done in another branch does not change `main` until they are joined.



// branches

Changing branch with git switch

`git switch` changes from one branch to another. Since each commit stores a **full snapshot** of the files (lesson 7), Git swaps the contents of the folder for those of the latest commit of the chosen branch, by itself.

With `git switch main` (`7c41af8`)

index.html

```
1  <title>Ana Silva</title>
2  …
3  <h2>Interests</h2>
```

With `git switch about-me` (`2d6b0e4`)

index.html

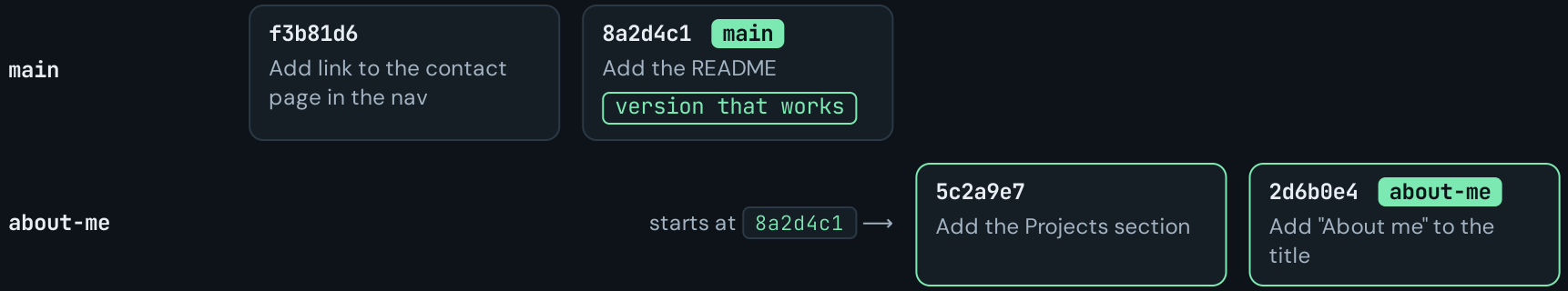
```
1  <title>About me · Ana Silva</title>
2  …
3  <h2>Interests</h2>
4  <h2>Projects</h2>
```

It is the same file, in the same folder. The title changes, and the Projects section only exists on `about-me`.

// branches

The history gets branches

With branches, the history stops being a single line and gets **branches**, like a tree. Each branch has its own commits, and the label with the branch name is on the latest one.

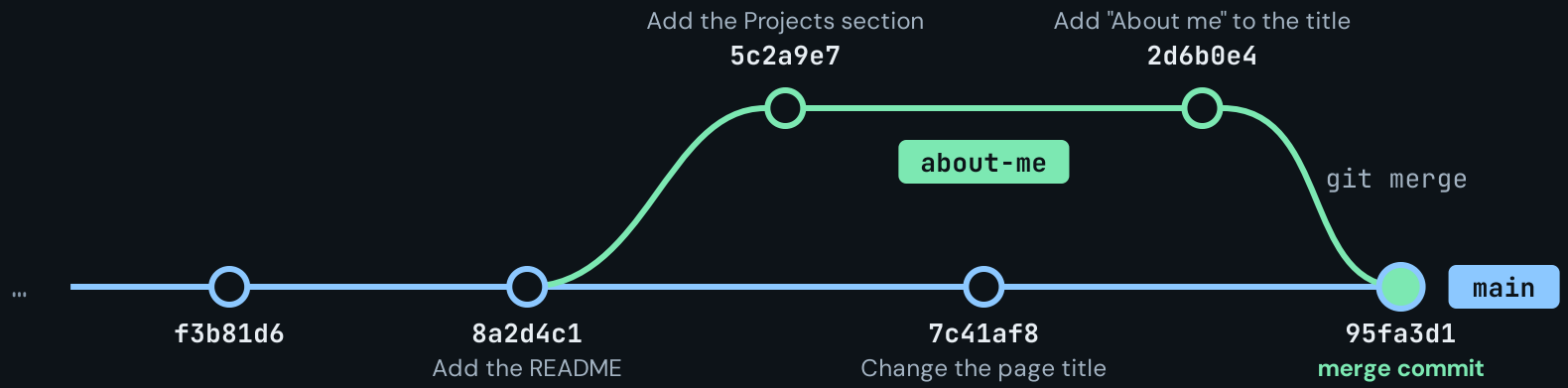


> **TIP** `git switch`, for changing branch, arrived in Git 2.23, in 2019. Since 2020, GitHub has called the main branch `main`, instead of `master`. More in the Pro Git chapter on branches.

// merge

The branches come back together

When the work in a branch is ready, it is joined into `main` with a **merge**. The two branches become one again, and `main` gets the commits from both.



// merge

Joining a branch with git merge

`git merge` brings the commits of another branch into the current branch. If `main` has not changed in the meantime, Git just moves the `main` label forward, which is called a **fast-forward**. If both have changed, Git creates a **merge commit**, which joins the two lines.



// conflicts

What if both change the same line?

While the `about-me` branch was not ready, Ana changed the page title on `main`. Both branches changed the same line of `index.html`, each in its own way.

on `main`, commit `7c41af8`

```
<title>Ana Silva</title>
```

on `about-me`, commit `2d6b0e4`

```
<title>About me · Ana Silva</title>
```

`git merge` →

result

?

// conflicts

Resolving a conflict

Git does not choose by itself. It marks the **conflict** in the file, with both versions, and the merge stops until it is resolved.

01 Keep only the right version and delete the three marker lines.

02 Save and run `git add index.html`.

03 Finish the merge with `git commit -m "Merge the about-me branch"`.

Between `<<<<<<< HEAD` and `=======` is the version of the current branch, `main`. Between `=======` and `>>>>>>>`, the one from the branch coming in.

index.html

```
1 <<<<<<< HEAD
2   <title>Ana Silva</title>
3 =======
4   <title>About me · Ana Silva</title>
5 >>>>>>> about-me
```

`git status` shows the conflicting file as "both modified".

// commands

Branch commands

COMMAND	WHAT IT IS FOR
`git branch`	lists the branches; the current one has `*`
`git switch -c about-me`	creates a branch and switches to it
`git switch main`	switches to the `main` branch
`git merge about-me`	joins that branch into the current one
`git log --oneline --graph`	shows the history, with the branches

WARNING Before changing branch, commit everything, until `git status` says "nothing to commit".

01 · activity

Activity 1

Experimenting in a branch

12 min · individual

// activity 1

Experimenting in a branch

01 Create the branch with `git switch -c about-me`.

02 In `index.html`, change the `<title>` and add a new section, with an `<h2>`, and commit.

03 Go back to `main` with `git switch main` and, to cause a conflict, change the same `<title>` in another way and commit.

04 Run `git merge about-me`, resolve the conflict and finish with `git add` and `git commit`.

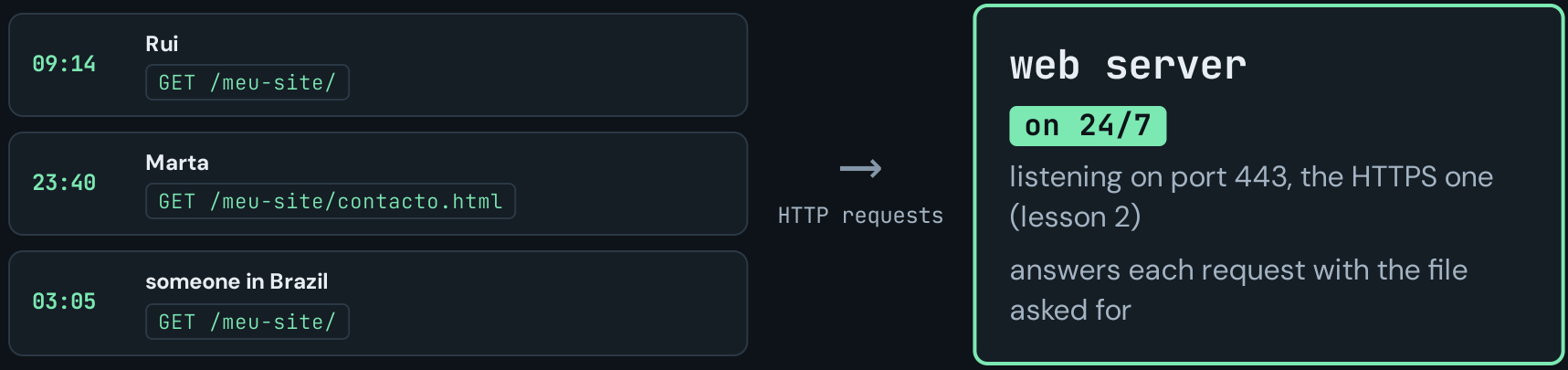
05 Send it with `git push` and see on GitHub the commits from both branches.

> Finished? See the two lines of the history with `git log --oneline --graph`.

// server

A site needs a server that listens

For a page to be available, there has to be a **web server** that is on 24 hours a day, listening for HTTP requests (lesson 3). That way, anyone who asks for the page gets an answer, at any time and from anywhere.



// the problem

The repository is not the site

Anyone who opens `github.com/ana-silva/meu-site` sees the **code** and the list of files. Someone visiting a site expects to see the **page**, as the browser shows it.

what the repository shows

ana-silva / meu-site Public

Merge the about-me branch	95fa3d1 · 9 commits
README.md	Add the README
contacto.html	Add link to the contact page in the nav
foto.jpg	Add the about me page
index.html	Merge the about-me branch

Mini-site

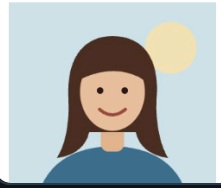
Ana Silva's site, with an "about me" page and a contact form.

what we want to show

ana-silva.github.io/meu-site/

- About me
- Contact

About me



// hosting

Hosting and GitHub Pages

Keeping a server on at home, 24 hours a day, is expensive and hard. So a **hosting** service is used, which keeps the site on its servers and answers the requests.



GitHub Pages, which has existed since December 2008, is free hosting for static sites (HTML, CSS and JavaScript, with no back end), published from a repository.

// address

The site’s address

The address reads like any URL (lesson 2). The username becomes a **subdomain** and the repository name becomes the **path**.

```
https://ana-silva.github.io/meu-site/
```

`https://`
the scheme, the secure protocol

`.github.io`
the domain, the same for every GitHub Pages site

`ana-silva`
the subdomain, which is the GitHub username

`/meu-site/`
the path, which is the repository name

Each file has its own address, such as `…/meu-site/contacto.html`. Without a file name, the server delivers `index.html`.

// enable

Turning on GitHub Pages

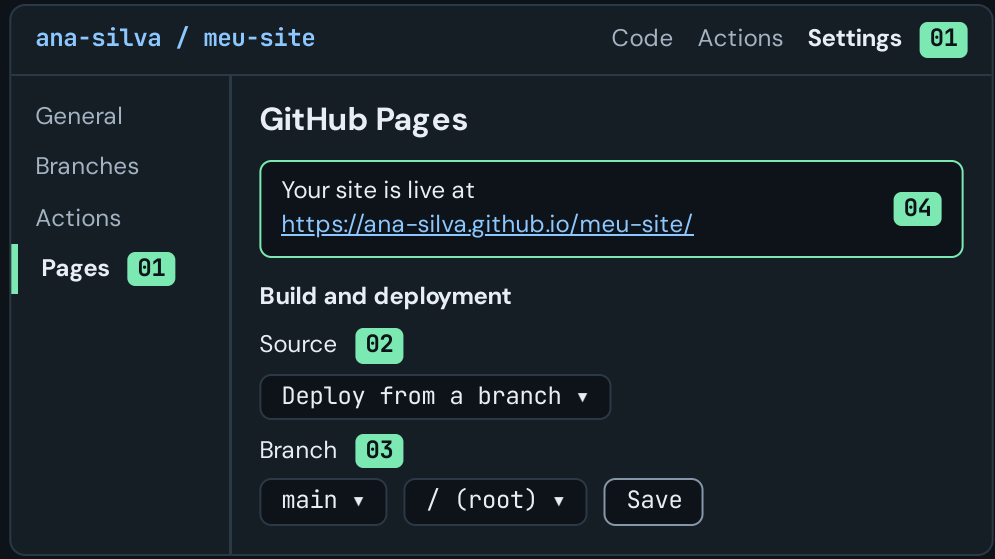
01 In the repository, open **Settings** and, in the sidebar, **Pages**.

02 Under "Source", choose "Deploy from a branch".

03 Choose `main` and `/ (root)` and click **Save**.

04 Wait about a minute, reload, and open the address under "Your site is live at".

More in the GitHub Pages documentation.



// publishing

Every push publishes the site again

Once it is on, GitHub Pages publishes the site again on every `git push` to `main`, the only branch that is published. Each publish is called a **deploy**.



// question

Why doesn’t the photo show up?

On Ana’s laptop, the photo shows up. On GitHub Pages, with the same HTML files, it does not.

on Ana's laptop

file:///C:/Users/ana/…/index.html

- About me
- Contact

About me



CTeSP DTAM student.

Interests

on GitHub Pages

ana-silva.github.io/meu-site/

- About me
- Contact

About me

Portrait of Ana

CTeSP DTAM student.

Interests

cause

?

// common mistakes

Common publishing mistakes

If the requested file does not exist on the server, the HTTP response is **404** (not found), a 4xx code from lesson 3. An image with a 404 shows up broken.

CAUSE	FIX
`foto.jpg` never went into a commit and shows under "Untracked files"	`git add foto.jpg`, commit and push
a path on the disk, such as `src="C:\Users\ana\Desktop\foto.jpg"`	a relative path, `src="foto.jpg"`
`src="/foto.jpg"`, which points to `ana-silva.github.io/foto.jpg`	remove the slash at the start
`Foto.JPG` on the disk and `foto.jpg` in the HTML, which the laptop accepts and the server does not	the same name in both, in lower case
`index.html` with another name or in a subfolder	`index.html` at the root

02 · activity

Activity 2

Publishing the site

16 min · individual

// activity 2

Publishing the site

01 Turn on GitHub Pages in the `meu-site` repository (**Settings → Pages**).

02 Open the published address and test every page, link and image.

03 Put the address in the repository's "About" (gear icon → "Use your GitHub Pages website").

04 Make a small change, commit and `git push`, and check that the published site changes.

> TIP If the page does not change after the deploy, reload without the cache, with `Ctrl` + `F5` (`Cmd` + `Shift` + `R` on macOS).

// who uses it

Can anyone use the site?

Ana's site is on the Internet, and anyone with the address can **open** it. Opening it is not the same as being able to **use** it.

`ana-silva.github.io/meu-site/` → ? ? ?

Who might find it hard to use the site, and why?

// who uses it

Who uses Ana’s site

no limitation	permanent	temporary	permanent	situational
Ana	**Marta**	**Rui**	**Ana's grandmother**	**on the metro**
sees, hears and uses the mouse, and also reads a well-organised page more easily	is blind and listens to the page with a screen reader	broke his arm and, for a few weeks, only uses the keyboard	cannot see well up close and makes the text bigger with zoom	without sound, anyone watches a video with captions

Web **accessibility** means making sites that everyone can use, including people with disabilities. A limitation can be permanent, temporary or situational.

The three types come from Microsoft's Inclusive Design Toolkit (2016).

// why

Why accessibility matters

Accessibility is a right of people with disabilities and, in Portugal, a **legal obligation**.

1 in 6

About 1.3 billion people have a significant disability (WHO, 2023).

DL 83/2018

Public sector websites and apps have to be accessible.

DL 82/2022

Since 28 June 2025, also services such as online shopping and banking.

The technical rules these laws require come from **WCAG**, which comes up later.

// keyboard

Getting around with the keyboard only

People who do not use a mouse, like Rui, get around with the keyboard. The **focus** is the element that receives the keys, marked with an outline.

KEY	WHAT IT DOES
`Tab`	moves to the next element
`Shift` + `Tab`	goes back one element
`Enter`	follows a link, presses a button
`Space`	presses a button, ticks a box

`01` `link` About me

`02` `link` Contact

`03` `field` Name

`04` `field` Email

`05` `list` Subject

`06` `field` Message

`07` `box` I agree to be contacted

`08` `button` Send

The focus order in `contacto.html` follows the **HTML order**. A `<div>` with `onclick` does not get the focus (lesson 6).

03 · activity

Activity 3

Keyboard only

10 min · individual

// activity 3

Keyboard only

01 Open your own published site and, from then on, do not touch the mouse.

02 With `Tab`, reach every link and follow it with `Enter`.

03 In `contacto.html`, reach every field, fill it in and reach the send button.

04 Go back to the start of the page with `Shift` + `Tab`.

05 Note where the focus gets lost, skips an element or stops being visible.

> The notes are for activity 4.

// history

Where the rules come from

In 1997, the W3C (lesson 3) created the **WAI**, the Web Accessibility Initiative, to make the Web accessible to people with disabilities. The WAI writes **WCAG**, the accessibility guidelines for web content.

- **1997** — the WAI is created
- **1999** — WCAG 1.0
- **2008** — WCAG 2.0
- **2018** — WCAG 2.1
- **2023** — WCAG 2.2, the current one

> "The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect."
> Tim Berners-Lee, 1997

More on the W3C WAI website.

// wcag

The four principles of WCAG

WCAG rests on four principles, known by the acronym **POUR**. Each one has already come up in earlier lessons.

perceivable

The information reaches people who cannot see or hear it, like the `alt` of an image (lesson 4).

operable

Everything works with the keyboard, like a `<button>` instead of a `<div>` (lesson 6).

understandable

The content and the controls make sense, as with `<label>` and `lang` (lessons 5 and 6).

robust

The HTML is valid and semantic, and works in browsers and screen readers.

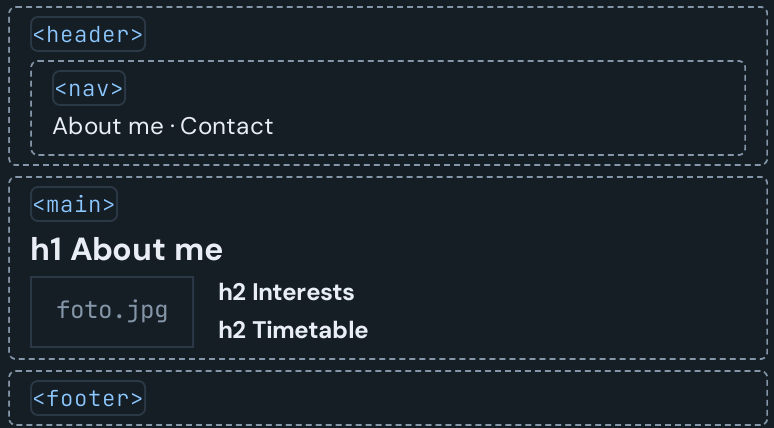
Each criterion has a level, A, AA or AAA. **Level AA**, which includes A, is what the law requires. Full text in WCAG 2.2.

// structure

The map of the page

The `<header>`, `<nav>`, `<main>` and `<footer>` elements mark the **regions** of the page, and the headings, from `<h1>` to `<h6>`, mark the sections. The screen reader shows them as a **map**.

index.html in the browser



what the screen reader shows

regions

- banner (`header`)
- navigation (`nav`)
- main content (`main`)
- footer (`footer`)

headings

- level 1 · About me
  - level 2 · Interests
  - level 2 · Timetable

// alt text

How to write alternative text

WRONG	`alt="photo"`	does not say what the image shows
WRONG	`alt="foto.jpg"`	the file name describes nothing
AVOID	`alt="Image of a girl"`	repeats "image", does not say who
RIGHT	`alt="Portrait of Ana"`	says what the image shows
RIGHT	`alt=""`	purely decorative, the reader skips it

// screen reader

The screen reader

A **screen reader** reads the page aloud or in braille and moves through its structure, by headings, regions and links.

- **NVDA**, on Windows, free and open source
- **VoiceOver**, on macOS and iOS
- Narrator, on Windows, and TalkBack, on Android

NVDA download page

ACTION	NVDA	VOICEOVER
turn on or off	`Ctrl` + `Alt` + `N` · NVDA+ `Q`	`Cmd` + `F5`
next heading	`H`	VO+ `Cmd` + `H`
next region	`D`	in the rotor
next link	`K`	VO+ `Cmd` + `L`
elements list	NVDA+ `F7`	rotor, VO+ `U`

The NVDA key is `Insert`, or `Caps Lock` if that option is chosen, useful on laptops. VO is `Ctrl` + `Option`. The `Ctrl` key stops the speech in both.

04 · activity

Activity 4

Fixing in a branch and publishing

12 min · individual

// activity 4

Fixing in a branch and publishing

01 Create the branch with `git switch -c accessibility`.

02 Fix an accessibility problem on the site, such as the ones noted in activity 3, look at it with `git diff` and commit.

03 Go back to `main` with `git switch main` and run `git merge accessibility`.

04 Send it with `git push`.

05 Wait for the deploy and check the fix on the published site, with the keyboard or the screen reader.

> Finished? Fix another problem, in a new branch.

UTPORTO
Universidade Técnica do Porto

NORTE 2030
Programa Regional do Norte
PORTUGAL 2030
Cofinanciado pela União Europeia

// end

Next lesson

CSS I, with selectors, the cascade and the box model. The published site will get styles, and every `git push` publishes it again.

Until then, have the site published, with at least one accessibility fix made in a branch.

Lecturer
Gonçalo Reais